Machine Learning

Machine learning with scikit-learn — Lecture 19 (Chapter 9, Part 2)

Amir Farbin

Part 2 — Machine learning with scikit-learn

Why a framework

- We built Fisher's discriminant **by hand**: none of it is difficult
- A framework makes it **fast**, and lets you **swap algorithms** with the same few lines, so you can try several and **compare** them
- **scikit-learn** is the standard for this kind of machine learning
- Its documentation is organised by **task**: classification, regression, clustering, dimensionality reduction, model selection, metrics, preprocessing, pipelines

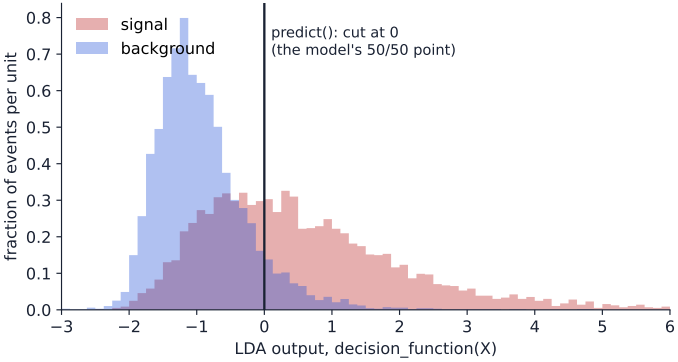
One interface, many algorithms

- Every model is a **class**: make an **instance**, then
  - `fit(X, y)`: learn the parameters, which stay in that instance
  - `decision_function(X)`: the continuous output
  - `predict(X)`: the output, cut at a default threshold
- A **pipeline** chains steps (scale, reduce, classify) without knowing which model it holds: **polymorphism**, as in Chapter 4's gradebook

Formulating the problem

- **X:** the 18 features. **y:** the `signal` column
- **Never give the answer as an input.** A score of 100% means: look for the answer, or something perfectly correlated with it, inside X
- Train on 4 million events, test on 1 million. Splitting is just slicing (`train_test_split` adds shuffling)
- Rates are per class: TPR on signal only, FPR on background only

LDA is Lecture 17's Fisher discriminant

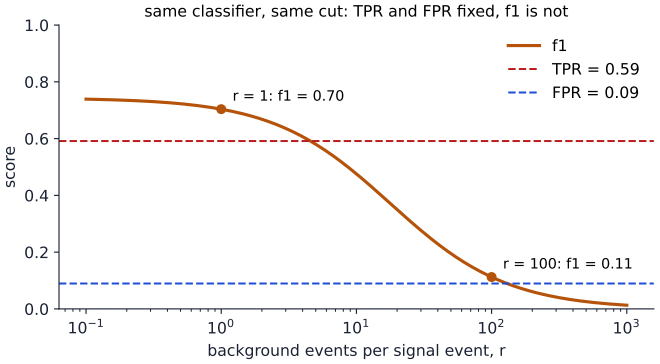


- Same direction as our hand-built w, same AUC. Its zero is the model's **50/50 point**
- `predict` = `decision_function > 0`. A default, **not** the best cut

ROC and AUC from the library

- `roc_curve(y, score)`, `auc(fpr, tpr)`: what you wrote in Lecture 17
- Give it the **score**, not `predict`: 0/1 predictions make a two-segment "curve"
- The thresholds it returns turn "I want this TPR" into a cut
- AUC on train vs test: **0.837 vs 0.838**, so no overtraining (18 weights, millions of events)

TPR and FPR don't depend on the mixture. f1 does



- $f_1 = 2\,\mathrm{TPR}/\left(2\,\mathrm{TPR} + (1-\mathrm{TPR}) + r\,\mathrm{FPR}\right)$ for r background per signal

Tuning needs its own sample

- Picking the threshold that maximises f1 **on the test sample** is fitting to the test sample
- Tune on **validation**, report once on **test**
- The same goes for choosing between algorithms, and their settings

scikit-learn's own examples

- Standard datasets for learning and benchmarking: **iris** (150 flowers, 4 measurements) and **digits** (1,797 images, $8 \times 8 = 64$ numbers)
- `SVC` on digits: the same pattern, instance, `fit`, `predict`
- One correct prediction is not a test: on 297 held-out digits it gets **95%**
- **Lab 8:** three algorithms on SUSY, compare ROC curves, carry the best to a significance